

Comprehensive MySQL Guidebook

A Practical YouTube Course Companion | Sample Database: mystore

DATABASE SCHEMA REFERENCE: MYSTORE

categories - category_id (PK) - category_name	products - product_id (PK) - product_name - category_id (FK) - price - stock_quantity	customers - customer_id (PK) - full_name - email (UNIQUE) - city - joined_date	orders - order_id (PK) - customer_id (FK) - product_id (FK) - order_date - quantity - total_amount
--	--	---	---

Module 1: Data Definition Language (DDL)

1. Database Management (`CREATE`, `USE`, `DROP`)

DDL commands define and alter structure. Before writing queries, you must create and select your target database environment.

Example 1: Create a new database safely

```
CREATE DATABASE IF NOT EXISTS mystore;
```

Prevents errors if the database already exists.

Example 2: Switch context to the database

```
USE mystore;
```

Sets mystore as the active working database for subsequent operations.

Example 3: Drop a database and cleanup

```
DROP DATABASE IF EXISTS old_mystore_backup;
```

Permanently deletes the specified database and all tables inside it.

2. Table Creation & Constraints (`CREATE TABLE`)

Tables store records. Constraints like PRIMARY KEY, FOREIGN KEY, NOT NULL, and UNIQUE preserve data integrity.

Example 1: Create the Customers Table

```
CREATE TABLE customers (  
  customer_id INT AUTO_INCREMENT PRIMARY KEY,  
  full_name VARCHAR(100) NOT NULL,  
  email VARCHAR(100) NOT NULL UNIQUE,  
  city VARCHAR(100) DEFAULT NULL,  
  joined_date DATE NOT NULL  
);
```

Defines primary keys and unique constraints to prevent duplicate emails.

Example 2: Create Products Table with Foreign Key

```
CREATE TABLE products (
  product_id INT AUTO_INCREMENT PRIMARY KEY,
  product_name VARCHAR(100) NOT NULL,
  category_id INT,
  price DECIMAL(10,2) NOT NULL,
  stock_quantity INT DEFAULT 0,
  FOREIGN KEY (category_id) REFERENCES categories(category_id) ON DELETE SET NULL
);
```

Ensures relational integrity between products and categories.

Example 3: Create Orders Table with Cascade Rules

```
CREATE TABLE orders (
  order_id INT AUTO_INCREMENT PRIMARY KEY,
  customer_id INT NOT NULL,
  product_id INT NOT NULL,
  order_date DATE NOT NULL,
  quantity INT NOT NULL,
  total_amount DECIMAL(10,2) NOT NULL,
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id) ON DELETE CASCADE,
  FOREIGN KEY (product_id) REFERENCES products(product_id) ON DELETE CASCADE
);
```

Automatically deletes child orders when the referenced customer/product is deleted.

3. Modifying & Dropping Structures (`ALTER`, `DROP`)

Modify existing schema structures dynamically using ALTER TABLE without losing data.

Example 1: Add a new column to Customers

```
ALTER TABLE customers ADD COLUMN phone_number VARCHAR(15) AFTER email;
```

Inserts a new contact phone number field right after the email column.

Example 2: Modify column data type & default

```
ALTER TABLE products MODIFY COLUMN stock_quantity INT NOT NULL DEFAULT 10;
```

Changes column constraints to prevent NULLs and sets a baseline stock value.

Example 3: Drop an unnecessary column or table

```
ALTER TABLE customers DROP COLUMN phone_number;
DROP TABLE IF EXISTS temporary_logs;
```

Removes schema attributes or obsolete database objects.

Module 2: Data Manipulation Language (DML)

1. Inserting Data (`INSERT INTO`)

Insert single or multiple rows into database tables.

Example 1: Single Row Insert into Categories

```
INSERT INTO categories (category_name) VALUES ('Laptops');
```

Inserts one new record. Auto-increment assigns the next available ID.

Example 2: Bulk Multi-Row Insert into Products

```
INSERT INTO products (product_name, category_id, price, stock_quantity) VALUES
('ProBook 15', 1, 65000.00, 50),
('Gaming Laptop X', 1, 85000.00, 30),
('Wireless Earbuds', 2, 3500.00, 150);
```

Efficiently adds multiple records in a single execution step.

Example 3: Insert with Default Value Fallback

```
INSERT INTO customers (full_name, email, city, joined_date)
VALUES ('Ariful Mondal', 'ariful@example.com', DEFAULT, '2024-03-01');
```

Uses table definition default ('NULL') for the city column.

2. Updating & Deleting Data ('UPDATE', 'DELETE')

Modify existing records or purge unwanted data. Always use WHERE clause to prevent unintended mass modifications!

Example 1: Update specific product price

```
UPDATE products
SET price = 62000.00, stock_quantity = stock_quantity + 10
WHERE product_id = 1;
```

Adjusts pricing and increments stock count for 'ProBook 15'.

Example 2: Delete a single record safely

```
DELETE FROM customers WHERE customer_id = 15;
```

Removes customer 15. Associated orders are cascadingly removed.

Example 3: Conditional bulk update

```
UPDATE products
SET stock_quantity = 100
WHERE category_id = 2;
```

Updates stock for all products under Audio category (category_id = 2).

Module 3: Basic Data Querying & Filtering

1. Selecting Data & Aliases ('SELECT', 'AS')

Retrieve specific columns, perform calculations, and alias table attributes for clarity.

Example 1: Simple Column Projection & Alias

```
SELECT full_name AS customer_name, email AS contact_email FROM customers;
```

Renames headers in output for cleaner display.

Example 2: Calculated Columns

```
SELECT product_name, price, stock_quantity,
       (price * stock_quantity) AS total_inventory_value
FROM products;
```

Computes inventory value dynamically per product line item.

Example 3: Distinct Selection

```
SELECT DISTINCT city FROM customers WHERE city IS NOT NULL;
```

Filters out duplicate city values from customer demographics.

2. Filtering Records (`WHERE`, Operators, `LIKE`, `IN`, `BETWEEN`)

Filter rows using comparisons, range operators, pattern matching, and logical conditions.

Example 1: Range Filtering (`BETWEEN`)

```
SELECT * FROM products WHERE price BETWEEN 10000.00 AND 50000.00;
```

Retrieves mid-tier products priced inclusively between 10k and 50k.

Example 2: Pattern Matching (`LIKE` / Wildcards)

```
SELECT * FROM customers WHERE full_name LIKE '%Sharma%' OR email LIKE '%@example.com';
```

Finds customers with 'Sharma' anywhere in their full name.

Example 3: Membership List Filtering (`IN`)

```
SELECT * FROM customers WHERE city IN ('Mumbai', 'Delhi', 'Bangalore');
```

Shorthand for multiple OR conditions across target metro cities.

3. Sorting & Pagination (`ORDER BY`, `LIMIT`)

Sort result sets ascending (`ASC`) or descending (`DESC`) and limit output size.

Example 1: Top 5 Highest Value Orders

```
SELECT * FROM orders ORDER BY total_amount DESC LIMIT 5;
```

Sorts total order value descending and yields the top 5 transactions.

Example 2: Multi-Level Sorting

```
SELECT * FROM customers ORDER BY city ASC, full_name ASC;
```

Orders primary records alphabetically by city, then secondary by customer name.

Example 3: Pagination with `LIMIT` & `OFFSET`

```
SELECT * FROM orders ORDER BY order_date DESC LIMIT 10 OFFSET 20;
```

Fetches page 3 of results (records 21–30) sorted by order date.

Module 4: Aggregation & Grouping

1. Aggregate Functions (`COUNT`, `SUM`, `AVG`, `MIN`, `MAX`)

Summarize dataset metrics across entire tables or filtered subgroups.

Example 1: High-level Revenue Summaries

```
SELECT COUNT(*) AS total_orders,  
       SUM(total_amount) AS total_revenue,
```

```
AVG(total_amount) AS avg_order_value
FROM orders;
```

Calculates core KPIs across all orders in the mystore dataset.

Example 2: Price Extremes in Inventory

```
SELECT MIN(price) AS cheapest_item, MAX(price) AS priciest_item FROM products;
```

Identifies minimum and maximum product list prices.

Example 3: Counting Non-NULL Values

```
SELECT COUNT(city) AS customer_cities_recorded FROM customers;
```

Counts only rows where city attribute is populated (ignoring NULLs).

2. Grouping & Filtering Groups (`GROUP BY`, `HAVING`)

Segment rows into subsets and filter aggregated group outputs using HAVING (not WHERE).

Example 1: Sales Summary by Customer

```
SELECT customer_id, COUNT(order_id) AS order_count, SUM(total_amount) AS total_spent
FROM orders
GROUP BY customer_id;
```

Aggregates sales metrics per distinct customer ID.

Example 2: Filter VIP Customers (`HAVING`)

```
SELECT customer_id, SUM(total_amount) AS total_spent
FROM orders
GROUP BY customer_id
HAVING total_spent > 100000.00;
```

Filters groups post-aggregation to highlight buyers who spent over 100,000.

Example 3: City Demographics Distribution

```
SELECT city, COUNT(customer_id) AS total_customers
FROM customers
GROUP BY city
HAVING total_customers >= 2
ORDER BY total_customers DESC;
```

Groups customers by city and shows locations with at least 2 registered customers.

Module 5: Multi-Table Relational Joins

1. Inner & Outer Joins (`INNER`, `LEFT`, `RIGHT`)

Combine fields from multiple tables based on related foreign key attributes.

Example 1: INNER JOIN (Orders with Customer Details)

```
SELECT o.order_id, c.full_name, c.email, o.order_date, o.total_amount
FROM orders o
INNER JOIN customers c ON o.customer_id = c.customer_id
LIMIT 5;
```

Matches records existing in both orders and customers tables.

Example 2: LEFT JOIN (Products with Categories)

```
SELECT p.product_name, p.price, c.category_name
FROM products p
LEFT JOIN categories c ON p.category_id = c.category_id;
```

Includes ALL products, even if category_id is NULL or invalid (like product ID 7).

Example 3: Multi-Table JOIN (Orders, Customers, Products)

```
SELECT o.order_id, c.full_name AS customer_name, p.product_name, o.quantity, o.total_amount
FROM orders o
INNER JOIN customers c ON o.customer_id = c.customer_id
INNER JOIN products p ON o.product_id = p.product_id
ORDER BY o.order_date DESC LIMIT 5;
```

Joins 3 tables to give full detailed order invoice overview.

Module 6: Advanced Subqueries & Functions

1. Subqueries & Nested Statements

Nest queries inside WHERE, FROM, or SELECT blocks to solve complex dynamic questions.

Example 1: Scalar Subquery in WHERE

```
SELECT product_name, price FROM products
WHERE price > (SELECT AVG(price) FROM products);
```

Finds products priced higher than the average catalog price.

Example 2: Multi-row Subquery ('IN')

```
SELECT full_name, email FROM customers
WHERE customer_id IN (
    SELECT DISTINCT customer_id FROM orders WHERE total_amount > 80000.00
);
```

Finds customer details for anyone who placed a single order over 80,000.

Example 3: Correlated Subquery ('EXISTS')

```
SELECT c.category_name FROM categories c
WHERE EXISTS (
    SELECT 1 FROM products p WHERE p.category_id = c.category_id
);
```

Returns categories that have at least one product mapped to them.

2. String & Date Built-In Functions

Manipulate string text and perform date arithmetic natively in MySQL.

Example 1: String Manipulation Functions

```
SELECT UPPER(full_name) AS uppercase_name,
    CONCAT(full_name, ' (', city, ')') AS customer_label
FROM customers LIMIT 5;
```

Formats names into uppercase and concatenates city attributes.

Example 2: Date Extraction (`YEAR`, `MONTH`, `DATE\_FORMAT`)

```
SELECT order_id, order_date,  
       YEAR(order_date) AS order_year,  
       MONTHNAME(order_date) AS order_month  
FROM orders LIMIT 5;
```

Extracts calendar elements from temporal order dates.

Example 3: Conditional Formatting (`CASE WHEN`)

```
SELECT product_name, stock_quantity,  
       CASE  
         WHEN stock_quantity < 50 THEN 'Low Stock - Reorder'  
         WHEN stock_quantity BETWEEN 50 AND 100 THEN 'Moderate'  
         ELSE 'Sufficient Stock'  
       END AS inventory_status  
FROM products;
```

Applies logic checks directly within standard SQL select queries.

Module 7: Database Administration & Optimization

1. Views, Indexing & Performance

Optimize query execution and simplify multi-table queries using database views and indexes.

Example 1: Creating a Saved View (`CREATE VIEW`)

```
CREATE VIEW customer_order_summary AS  
SELECT c.customer_id, c.full_name, COUNT(o.order_id) AS total_orders, SUM(o.total_amount) AS  
total_spent  
FROM customers c  
LEFT JOIN orders o ON c.customer_id = o.customer_id  
GROUP BY c.customer_id, c.full_name;
```

Saves dynamic query logic as a virtual view table for reusable reporting.

Example 2: Creating an Index for Fast Lookups

```
CREATE INDEX idx_orders_customer_date ON orders(customer_id, order_date);
```

Speeds up range filters and join conditions on customer orders.

Example 3: Query Execution Analysis (`EXPLAIN`)

```
EXPLAIN SELECT * FROM orders WHERE customer_id = 5 AND order_date >= '2024-05-01';
```

Inspects query optimizer execution plan to verify index utilization.